

Przewodnik egzaminacyjny Java — Część 1: Klasy abstrakcyjne (pytania 1-40)

1. Klasy abstrakcyjne — fundament koncepcji

Co to jest klasa abstrakcyjna?

Klasa abstrakcyjna to klasa oznaczona modyfikatorem `abstract`, której **nie można bezpośrednio instancjonować** (nie można wywołać `new NazwaKlasy()`). Jej rolą jest stanowienie szablonu — definiuje kontrakt i wspólne zachowania dla klas potomnych.

```
java

public abstract class Animal {
    // pole zwykłe — dozwolone
    private String name;

    // konstruktor — dozwolony (ale nie tworzy się obiektu Animal bezpośrednio)
    public Animal(String name) {
        this.name = name;
    }

    // metoda abstrakcyjna — bez ciała, MUSI być nadpisana w klasie konkretnej
    public abstract void makeSound();

    // metoda nieabstrakcyjna — ma ciało, jest dziedziczona
    public void sleep() {
        System.out.println(name + " is sleeping.");
    }
}
```

Kluczowe zasady:

- Klasa abstrakcyjna **może, ale nie musi** zawierać metod abstrakcyjnych.
- Klasa konkretna (nieabstrakcyjna) dziedzicząca po klasie abstrakcyjnej **musi zaimplementować wszystkie** metody abstrakcyjne — w przeciwnym razie sama musi być oznaczona jako `abstract`.
- Klasy abstrakcyjne mogą mieć pola, konstruktory, metody statyczne i niestatyczne.

Klasa abstrakcyjna a interfejs — kluczowe różnice

To jedno z najczęstszych pytań egzaminacyjnych. Zrozum je głęboko:

Cecha	Klasa abstrakcyjna	Interfejs (Java 8+)
Pola instancyjne	✅ Tak	❌ Nie (tylko <code>public static final</code>)
Konstruktory	✅ Tak	❌ Nie
Modyfikatory dostępu metod	Dowolne	Domyślnie <code>public</code> (od Java 9 — <code>private</code>)
Dziedziczenie	Jedna klasa (brak wielokrotnego)	Wiele interfejsów
Stan (state)	✅ Może przechowywać stan	❌ Nie przechowuje stanu
Metody z implementacją	✅ Dowolne	Od Java 8: <code>default</code> , <code>static</code> ; od Java 9: <code>private</code>

Komentarz: Wybierasz klasę abstrakcyjną gdy klasy potomne **muszą współdzielić stan** (pola instancyjne) lub gdy potrzebujesz konstruktorów z parametrami. Wybierasz interfejs, gdy chcesz zdefiniować kontrakt, który wiele niezwiązanych klas może spełnić. Kluczowe: klasa abstrakcyjna = "jest czymś" (is-a), interfejs = "potrafi coś" (can-do).

2. Hierarchia dziedziczenia i nadpisywanie metod

Pytanie 2 — dziedziczenie łańcuchowe

```
java

abstract class Animal {
    public abstract void makeSound();
    public void sleep() { System.out.println("Sleeping"); }
}

class Dog extends Animal {
    @Override
    public void makeSound() { System.out.println("Woof"); }
}

class GoldenRetriever extends Dog {
    // NIE nadpisuje makeSound()
}
```

`GoldenRetriever` dziedziczy implementację `makeSound()` z klasy `Dog`. Klasa `Dog` jako pierwsza dostarczyła konkretną implementację metody abstrakcyjnej — łańcuch jest

"zamknięty". `GoldenRetriever` **nie musi** jej nadpisywać, bo `Dog` nie jest abstrakcyjna. Wywoła się `Dog.makeSound()`.

Zasada ogólna: Metoda abstrakcyjna musi być zaimplementowana na pierwszym poziomie klasy **konkretnej** w hierarchii. Wszystkie klasy poniżej dziedziczą tę implementację, chyba że ją nadpiszą.

3. Modyfikatory i ich wpływ na kompilację

Niedozwolone kombinacje w metodach klasy abstrakcyjnej

```
java
abstract class Example {
    // ❌ BŁĄD KOMPILACJI – metoda prywatna nie może być abstrakcyjna
    // (klasy potomne nie mają do niej dostępu, więc nie mogą jej nadpisać)
    private abstract void doSomething();

    // ❌ BŁĄD KOMPILACJI – metoda statyczna nie może być abstrakcyjna
    // (metody statyczne nie są polimorficzne – należą do klasy, nie do instancji)
    static abstract void doStatic();

    // ❌ BŁĄD KOMPILACJI – final + abstract = sprzeczność logiczna
    // (final zabrania nadpisywania, abstract wymaga nadpisywania)
    final abstract void doFinal();

    // ✅ OK – protected abstract jest dozwolone
    protected abstract void doProtected();
}
```

Kombinacja `abstract` + `final` na poziomie KLASY:

```
java
// ❌ BŁĄD KOMPILACJI – klasa final nie może być abstract
public final abstract class MyClass { }
```

`final` zabrania dziedziczenia, `abstract` wymaga dziedziczenia (żeby ktoś mógł zaimplementować metody abstrakcyjne). Sprzeczność logiczna → błąd kompilacji.

4. Klasa abstrakcyjna bez metod abstrakcyjnych

To ważny przypadek brzegowy — klasa może być `abstract` bez żadnej metody abstrakcyjnej:

java

```
public abstract class BaseLogger {
    public void log(String message) {
        System.out.println("[LOG] " + message);
    }
    // brak metod abstrakcyjnych – to jest LEGALNE
}
```

Skutek: `BaseLogger` jest nadal klasa abstrakcyjna — **nie można jej instancjonować bezpośrednio**. Ale każda podklasa może być konkretna bez nadpisywania czegokolwiek. To wzorec przydatny gdy chcesz **zapobiec bezpośredniemu tworzeniu obiektów** klasy bazowej, nie wymuszając nadpisywania.

5. Konstruktory w klasach abstrakcyjnych

Klasy abstrakcyjne **mogą i często powinny** mieć konstruktory. Wywołuje się je przez `super()` z klasy potomnej.

java

```
abstract class Shape {
    private final String color;

    public Shape(String color) {
        this.color = color;
    }

    public String getColor() { return color; }
    public abstract double area();
}

class Circle extends Shape {
    private double radius;

    public Circle(String color, double radius) {
        super(color); // WYMAGANE – wywołanie konstruktora klasy abstrakcyjnej
        this.radius = radius;
    }

    @Override
    public double area() { return Math.PI * radius * radius; }
}
```

Pytanie 5 — klasa potomna bez konstruktora:

Jeśli klasa abstrakcyjna ma **tylko** konstruktor z parametrami (brak bezparametrowego), a klasa potomna nie definiuje konstruktora — kompilator próbuje wstawić domyślny konstruktor `super()`. Skoro brak konstruktora bezparametrowego w klasie bazowej → **błąd kompilacji**.

java

```
abstract class Base {
    Base(int x) { } // brak Base() bezparametrowego
}

class Child extends Base {
    // kompilator wstawiłby: Child() { super(); } – ale super() nie istnieje!
    // ❌ BŁĄD KOMPILACJI
}

class ChildFixed extends Base {
    ChildFixed(int x) { super(x); } // ✅ OK
}
```

6. Pola statyczne w klasach abstrakcyjnych

Pola statyczne w klasach abstrakcyjnych są jak w każdej klasie — są **współdzielone przez wszystkie instancje** (w tym instancje klas potomnych) i **dostępne bez tworzenia obiektu** przez nazwę klasy (lub klasy potomnej).

java

```
abstract class Counter {
    public static int count = 0; // wspólne dla wszystkich

    Counter() { count++; }
    public abstract void describe();
}

class Dog extends Counter {
    Dog() { super(); }
    public void describe() { System.out.println("Dog"); }
}
```

Ważne: `Counter.count` i `Dog.count` to **ta sama zmienna statyczna**.

7. Wzorzec Template Method

To jeden z klasycznych wzorców projektowych (GoF) i bardzo częste zastosowanie klas abstrakcyjnych.

```
java

abstract class DataProcessor {
    // METODA SZABLONOWA – final, żeby nikt jej nie zmienił
    public final void process() {
        readData();
        processData(); // wywołuje abstrakcyjną metodę!
        writeData();
    }

    private void readData() {
        System.out.println("Reading data...");
    }

    // Podklasy definiują ten krok
    protected abstract void processData();

    private void writeData() {
        System.out.println("Writing data...");
    }
}

class CSVProcessor extends DataProcessor {
    @Override
    protected void processData() {
        System.out.println("Processing CSV...");
    }
}
```

Kluczowa korzyść wzorca Template Method:

- Algorytm (kolejność kroków) jest zdefiniowany raz w klasie bazowej.
- Tylko specyficzne kroki są delegowane do podklas.
- Eliminuje duplikację kodu (DRY — Don't Repeat Yourself).

W pytaniu 7: `describe()` wywołuje `area()` — to jest właśnie Template Method. Gdy `describe()` jest wywołana na obiekcie `Circle`, przez polimorfizm zostanie wywołana `Circle.area()`.

8. Klasa abstrakcyjna implementująca interfejs (niekompletna implementacja)

```
java

interface Drawable {
    void draw();
    void resize(int factor);
}

// Klasa abstrakcyjna NIE musi implementować wszystkich metod interfejsu
abstract class AbstractShape implements Drawable {
    @Override
    public void draw() {
        System.out.println("Drawing shape");
    }
    // resize() nie jest zaimplementowane – to jest LEGALNE
    // Bo AbstractShape jest abstract – wymusza implementację na podklasach
}

class Square extends AbstractShape {
    @Override
    public void resize(int factor) { /* implementacja */ }
    // draw() jest dziedziczone z AbstractShape
}
```

Zasada: Klasa abstrakcyjna może "odroczyć" implementację metod interfejsu na swoje podklasy. To jest właśnie jeden z powodów, dla których klasa abstrakcyjna istnieje.

9. Wielokrotne dziedziczenie — ograniczenia Javy

Java **nie pozwala** na dziedziczenie po więcej niż jednej klasie (ani abstrakcyjnej, ani konkretnej). To celowe ograniczenie unikające problemu diamentu.

```
java

abstract class Vehicle { abstract void start(); }
abstract class Machine { abstract void start(); }

// ❌ NIEMOŻLIWE – Java nie wspiera wielokrotnego dziedziczenia klas
class Car extends Vehicle, Machine { }
```

Jak obejść? Interfejsy:

```
java
```

```

interface Startable { void start(); }

abstract class Vehicle implements Startable { }
abstract class Machine implements Startable { }

// Klasa Car może implementować wiele interfejsów
class Car extends Vehicle implements Startable {
    @Override
    public void start() { System.out.println("Car starting"); }
}

```

Lub wzorzec kompozycji (Car zawiera Vehicle i Machine jako pola).

10. Metoda final + metoda abstrakcyjna w tej samej klasie

```

java

abstract class AbstractTask {
    // final – podklasy NIE MOGĄ nadpisać
    public final void execute() {
        System.out.println("Starting task");
        doWork(); // delegowanie do abstrakcyjnej
        System.out.println("Task done");
    }

    // abstract – podklasy MUSZĄ nadpisać
    protected abstract void doWork();
}

```

Podklasa **musi** zaimplementować `doWork()`, ale **nie może** nadpisać `execute()`. To klasyczny Template Method z zabezpieczeniem `final`.

11. Chroniony konstruktor w klasie abstrakcyjnej

```

java

public abstract class AbstractProcessor {
    protected AbstractProcessor() {
        // dostępny tylko dla podklas i klas w tym samym pakiecie
    }
}

```


Modyfikator `protected` na konstruktorze klasy abstrakcyjnej to praktycznie standard. W praktyce:

- Klasy w **innym pakiecie** mogą tworzyć podklasy (przez `extends`) — konstruktor `protected` jest widoczny dla podklas niezależnie od pakietu.
 - Klasy w **innym pakiecie** NIE mogą wywołać konstruktora bezpośrednio (bo i tak nie można instancjonować klasy abstrakcyjnej).
-

12. Błędy czasu kompilacji vs czasu wykonania (generyki i polimorfizm)

W kontekście klas abstrakcyjnych, błąd kompilacji nastąpi gdy:

- Klasa konkretna nie implementuje wszystkich metod abstrakcyjnych.
- Próba stworzenia instancji klasy abstrakcyjnej (`new AbstractClass()`).
- Użycie niedozwolonych kombinacji modyfikatorów (`private abstract`, `final abstract`).

Błąd czasu wykonania (rzadszy w kontekście abstrakcji) — np. `ClassCastException` przy niepoprawnym rzutowaniu.

13. Polimorfizm — która metoda zostanie wywołana?

```
java

abstract class Animal {
    public void describe() {
        System.out.println("I am: " + getClass().getName());
        System.out.println("Sound: ");
        makeSound(); // late binding - wywoła metodę konkretnego obiektu
    }
    public abstract void makeSound();
}

class Cat extends Animal {
    @Override
    public void makeSound() { System.out.println("Meow"); }
}

// Użycie:
Animal a = new Cat(); // referencja typu Animal, obiekt Cat
a.describe();
// Wywoła: Cat.makeSound() - polimorfizm dynamiczny (late binding)
```

Kluczowy mechanizm: Java używa **dynamicznego wiązania** (dynamic dispatch / late binding) dla metod wirtualnych (niestatycznych, nieprywatnych, niefinalnych). Typ **referencji** decyduje co jest dostępne w czasie kompilacji, ale typ **obiektu** decyduje która implementacja zostanie wywołana w czasie wykonania.

14. Klasy anonimowe implementujące klasę abstrakcyjną

```
java

AbstractCache cache = new AbstractCache() {
    @Override
    public String get(String key) {
        return "value_" + key;
    }
    // getOrDefault() jest dziedziczone z AbstractCache
};

String result = cache.getOrDefault("myKey", "default");
// getOrDefault() wywoła this.get() – czyli naszą implementację z klasy anonimowej
```

Klasa anonimowa musi zaimplementować wszystkie metody abstrakcyjne. Może korzystać z nieabstrakcyjnych metod klasy bazowej (tu: `getOrDefault()`), które z kolei wywołują metody abstrakcyjne — i przez polimorfizm trafiają do implementacji w klasie anonimowej.

15. Łańcuch dziedziczenia a polimorfizm (zaawansowany)

```
java
```

```

abstract class Payment {
    public abstract boolean validate();

    public void process() {
        if (validate()) { // LATE BINDING – wywoła validate() konkretnego obiektu
            System.out.println("Processing payment...");
        }
    }
}

class CreditCardPayment extends Payment {
    @Override
    public boolean validate() { return true; }
}

class SecureCreditCardPayment extends CreditCardPayment {
    @Override
    public boolean validate() { return super.validate() && checkSecurity(); }
    private boolean checkSecurity() { return true; }
}

// Wywołanie:
Payment p = new SecureCreditCardPayment();
p.process();
// process() wywołuje validate() – obiekt jest SecureCreditCardPayment,
// więc wywoła SecureCreditCardPayment.validate()

```

Zasada: Java zawsze wywołuje metodę **najbardziej specyficzną** dla rzeczywistego typu obiektu w hierarchii dziedziczenia.

16. Poprawne kombinacje modyfikatorów na klasie

```

java

public abstract class MyClass { }           // ✅ OK
abstract class PackageClass { }             // ✅ OK – brak public = package-private
protected abstract class Inner { }          // ✅ OK dla klasy wewnętrznej (nested)

// ❌ Niedozwolone:
public final abstract class Bad { }          // final + abstract = sprzeczność
private abstract class AlsoBad { }           // private dla klas najwyższego poziomu = b

```

17. Wzorzec Decorator z klasą abstrakcyjną — cachowanie bez modyfikacji podklas

```
java

abstract class Repository {
    public abstract Object findById(int id);
}

// Dodajemy cachowanie przez NOWĄ klasę abstrakcyjną – wzorzec Decorator/Template Method
abstract class CachedRepository extends Repository {
    private Map<Integer, Object> cache = new HashMap<>();

    @Override
    public final Object findById(int id) {
        if (cache.containsKey(id)) {
            return cache.get(id);
        }
        Object result = fetchById(id); // delegowanie do nowej metody abstrakcyjnej
        cache.put(id, result);
        return result;
    }

    protected abstract Object fetchById(int id); // podklasy implementują to
}

class UserRepository extends CachedRepository {
    @Override
    protected Object fetchById(int id) {
        // pobieranie z bazy danych
        return new Object();
    }
}
```

To elegancki sposób na dodanie cachowania bez modyfikacji istniejących klas.

18. Klasa abstrakcyjna implementująca Runnable

```
java
```

```

abstract class AbstractTask implements Runnable {
    @Override
    public abstract void run(); // redekларacja jako abstrakcyjna – dozwolona
}

class ConcreteTask extends AbstractTask {
    @Override
    public void run() {
        System.out.println("Running task...");
    }
}

// Użycie z wątkiem:
ConcreteTask task = new ConcreteTask();
Thread thread = new Thread(task); // Thread akceptuje Runnable
thread.start(); // wywoła task.run()

```

Thread akceptuje `Runnable` — `ConcreteTask` jest `Runnable` (poprzez hierarchię dziedziczenia). Polimorfizm działa.

19. Refaktoryzacja — 9 z 10 klas ma identyczną implementację

To pytanie o zasadę DRY i dobre projektowanie:

Rozwiązanie: Jeśli 9 z 10 klas potomnych ma identyczną implementację `calculateFee()`, ta implementacja powinna być w klasie abstrakcyjnej jako metoda **nieabstrakcyjna** (domyślna implementacja). Jedna klasa specjalna po prostu ją nadpisze.

```

java

abstract class FeeCalculator {
    // Domyślna implementacja dla 9 klas
    public double calculateFee(double amount) {
        return amount * 0.02; // 2% standardowa opłata
    }
}

class SpecialCalculator extends FeeCalculator {
    @Override
    public double calculateFee(double amount) {
        return amount * 0.05; // specjalna opłata
    }
}

```

20. Serializacja i klasy abstrakcyjne

Klasa abstrakcyjna **może** implementować `Serializable`. Jeśli implementuje, to serializacja działa dla klas potomnych normalnie. Sama klasa abstrakcyjna nie może być serializowana (bo nie można stworzyć jej instancji).

```
java

abstract class AbstractEntity implements Serializable {
    private static final long serialVersionUID = 1L;
    private int id;
    // ...
}

class User extends AbstractEntity {
    private String name;
    // User jest serializowalny – dziedziczy Serializable z AbstractEntity
}
```

21. Prywatny konstruktor w klasie abstrakcyjnej — konsekwencje

```
java

abstract class Singleton {
    private Singleton() { } // JEDYNY konstruktor – prywatny
}
```

Konsekwencje:

- Klasy zewnętrzne nie mogą dziedziczyć po `Singleton` (bo konstruktor `super()` jest niedostępny poza klasą).
- Jedyna możliwość dziedziczenia to **klasy wewnętrzne (nested)** tej samej klasy.
- To de facto uniemożliwia dziedziczenie z zewnątrz, czyniąc klasę bezużyteczną jako klasa bazowa dla kodu zewnętrznego.

Architektonicznie: jeśli chcesz zapobiec dziedziczeniu, użyj `final`. Prywatny konstruktor w klasie abstrakcyjnej to dziwny wzorec — stosowany czasem do "zamknięcia" hierarchii (np. sealed classes w starszej Javie).

22. Dlaczego `static abstract` jest nielegalne?

To pytanie o fundamenty OOP:

```
java
// ✖ BŁĄD KOMPILACJI
abstract class Bad {
    static abstract void doSomething();
}
```

Przyczyna teoretyczna: Polimorfizm (mechanizm, który sprawia, że `abstract` ma sens) działa przez **dynamiczne wiązanie** oparte na typie obiektu w czasie wykonania. Metody statyczne są **wiązane statycznie** — w czasie kompilacji, na podstawie typu referencji, a nie obiektu. Nie istnieje "wirtualna tabela metod" (vtable) dla metod statycznych. Skoro nie można ich przesłonić (tylko ukryć, ang. hiding), nie ma sensu ich deklarować jako abstrakcyjne.

23. Deserializacja — klasa abstrakcyjna bez `Serializable` w hierarchii

```
java
abstract class Base {
    int x;
    // NIE implementuje Serializable
    public Base() { this.x = 42; } // musi mieć bezparametrowy konstruktor!
}

class Child extends Base implements Serializable {
    String name;
    // ...
}
```

Podczas **deserializacji** `Child`:

- Pola z `Child` (zdefiniowane w `Serializable` podklasie) są odtwarzane ze strumienia.
- Pola z `Base` (klasa nierzalizowalna) są inicjalizowane przez **bezparametrowy konstruktor** `Base()`.

Ważne: `Base` musi mieć dostępny bezparametrowy konstruktor — inaczej `InvalidClassException` podczas deserializacji.

24. Interfejs z `default` a klasa abstrakcyjna redefiniująca jako `abstract`

```
java

interface Drawable {
    default void draw() { System.out.println("Drawing"); }
}

abstract class AbstractShape implements Drawable {
    @Override
    public abstract void draw(); // "cofa" implementację default do statusu abstrak
}
```

Skutek: Klasy dziedziczące po `AbstractShape` muszą teraz zaimplementować `draw()`. Implementacja `default` z interfejsu jest "przykryta". To legalne i czasem użyteczne gdy chcesz wymusić własną implementację zamiast domyślnej.

25. Dlaczego `synchronized abstract` jest błędem semantycznym?

```
java

// ❌ BŁĄD KOMPILACJI
abstract class Worker {
    synchronized abstract void work();
}
```

Przyczyna: `synchronized` na metodzie instancyjnej blokuje monitor `this` — konkretnej instancji. Metoda abstrakcyjna **nie ma implementacji** — nie ma ciała, które mogłoby być wykonane pod blokadą. Synchronizacja musi być przypisana do konkretnej implementacji (w podklasie). Kompilator odmawia — to semantycznie bez sensu.

Poprawnie: podklasa sama decyduje o synchronizacji:

```
java

class ConcreteWorker extends Worker {
    @Override
    synchronized void work() { /* ... */ } // ✅ OK
}
```

26. Wzorzec Template Method — modyfikatory dostępu w praktyce

Typowy schemat:


```

java

abstract class AbstractReport {
    // Metoda szablonowa – public final
    public final void generate() {
        collectData();
        formatData();
        print();
    }

    // Kroki dla podklas – protected (dostępne dla potomków, ukryte dla klientów)
    protected abstract void collectData();
    protected abstract void formatData();

    // Krok wspólny – protected (podklasy mogą nadpisać, ale nie muszą)
    protected void print() {
        System.out.println("Printing report...");
    }
}

```

Typowe kombinacje:

- Metoda szablonowa: `public final`
- Kroki abstrakcyjne: `protected abstract`
- Kroki z domyślną implementacją: `protected` (opcjonalne nadpisanie)

`protected` ukrywa szczegóły implementacji przed klientami (wzorzec dobrego API).

27. Prywatna zagnieżdżona klasa abstrakcyjna

```

java

class Outer {
    // Prywatna zagnieżdżona klasa abstrakcyjna
    private abstract static class InnerAbstract {
        abstract void doWork();
    }

    // Tylko klasy wewnętrzne Outer mogą z niej korzystać
    private static class ConcreteInner extends InnerAbstract {
        @Override
        void doWork() { System.out.println("Working"); }
    }
}

```

Ograniczenia `private` na klasie wewnętrznej:

- Dostępna tylko wewnątrz `Outer`.
 - Kod poza `Outer` nie może dziedziczyć po niej ani jej używać.
 - Użyteczna do enkapsulacji wzorców projektowych wewnątrz klasy.
-

28. Deklaracja `public abstract String toString()`

```
java

abstract class MyClass {
    @Override
    public abstract String toString(); // redeklaracja toString() jako abstrakcyjne
}
```

`toString()` pochodzi z `java.lang.Object`. Deklarując ją jako abstrakcyjną wymuszamy na podklasach własną implementację (zamiast dziedziczenia domyślnej z `Object`). **Kompiluje się poprawnie.** To legalne i czasem sensowne w hierarchiach, gdzie domyślny `toString()` z `Object` byłby bezużyteczny.

29. Klasa anonimowa rozszerzająca klasę abstrakcyjną z konstruktorem z parametrami

```
java

abstract class Base {
    Base(int x) { System.out.println("Base(" + x + ")"); }
    abstract void work();
}

// Klasa anonimowa – jak wywołać konstruktor z parametrem?
Base b = new Base(42) { // wartość 42 jest przekazana do konstruktora Base
    @Override
    void work() { System.out.println("Working"); }
};
```

Ważne: Przy tworzeniu klasy anonimowej argumenty konstruktora podajesz w nawiasach po nazwie klasy. Kompilator przekazuje je do `super(...)`. Jeśli klasa abstrakcyjna ma TYLKO konstruktory z parametrami i nie podasz argumentów → błąd kompilacji.

30. Sealed abstract class (Java 17)

java

```
// Zapieczętowana hierarchia – tylko dozwolone podklasy mogą dziedziczyć
public sealed abstract class Shape permits Circle, Rectangle, Triangle {
    public abstract double area();
}

public final class Circle extends Shape {
    // ...
}
```

Co wyróżnia sealed classes:

- Kompilator **zna wszystkie możliwe podklasy** w czasie kompilacji.
- Umożliwia **wyczerpujące dopasowanie wzorców** (pattern matching) — kompilator może sprawdzić czy wszystkie przypadki są obsłużone.
- Hierarchia jest zamknięta — nikt z zewnątrz nie może dodać nowej podklasy.
- Podklasy muszą być w tym samym module (lub pakiecie).

To rozwiązanie łączy zalety enum (zamknięty zbiór wartości) z pełną mocą klas.

31. Kolejność bloków inicjalizacyjnych

java

```
abstract class Base {
    {
        System.out.println("Base instance initializer"); // 1
    }
    Base() {
        System.out.println("Base constructor"); // 2
    }
    abstract void work();
}

class Child extends Base {
    {
        System.out.println("Child instance initializer"); // 3
    }
    Child() {
        super(); // niejawne
        System.out.println("Child constructor"); // 4
    }
    void work() { }
}

new Child();
// Output: Base instance initializer → Base constructor → Child instance initializer
```

Zasada: Przy tworzeniu obiektu kolejność to zawsze:

1. Bloki inicjalizacyjne klasy bazowej (w kolejności w kodzie)
2. Konstruktor klasy bazowej
3. Bloki inicjalizacyjne klasy potomnej
4. Konstruktor klasy potomnej

32. Hiding metod statycznych vs overriding

java

```
abstract class Base {
    protected static void method() {
        System.out.println("Base.method()");
    }
}

class Child extends Base {
    protected static void method() { // To NIE jest override – to jest HIDING (ukrywanie)
        System.out.println("Child.method()");
    }
}

Base ref = new Child();
ref.method();    // UWAGA: wywoła Base.method()! – metody statyczne nie są polimorficzne
Child.method(); // wywoła Child.method()
```

Kluczowa różnica: Metody statyczne **nie są polimorficzne**. Java nazywa to "method hiding" (ukrywanie), nie "method overriding" (nadpisywanie). Typ **referencji** (nie obiektu) decyduje która statyczna metoda zostanie wywołana.

33. Kiedy klasa abstrakcyjna jest niezastąpiona?

Interfejsy w Java 8+ mogą mieć metody `default` i `static`, ale klasa abstrakcyjna wciąż jest niezastąpiona gdy:

1. **Potrzebujesz przechowywać stan** — pola instancyjne (interfejs może mieć tylko `static final`).
 2. **Potrzebujesz konstruktorów z parametrami** — interfejsy nie mają konstruktorów.
 3. **Potrzebujesz niestatycznych pól chronionych** — interfejsy nie mogą ich mieć.
 4. **Potrzebujesz różnych modyfikatorów dostępu dla metod** — w interfejsach metody są domyślnie `public`.
-

34. Checked exceptions w metodach abstrakcyjnych

```
java
```

```

abstract class FileProcessor {
    public abstract void process() throws IOException; // deklaracja wyjątku
}

class CSVProcessor extends FileProcessor {
    @Override
    // Podklasa może:
    // ✅ Rzucać IOException (taki sam jak w abstrakcyjnej)
    // ✅ Rzucać podklasę IOException (np. FileNotFoundException)
    // ✅ Nie rzucać wcale (zawęzić kontrakt)
    // ❌ Rzucać szerszy wyjątek (np. Exception) – BŁĄD KOMPILACJI
    public void process() throws FileNotFoundException { // OK – podklasa IOException
        // ...
    }
}

```

Zasada Liskov (LSP): Podklasa może "zawęzić" listę wyjątków (deklarować mniej lub bardziej szczegółowe), ale nie może "rozszerzyć" (deklarować wyjątków spoza hierarchii zadeklarowanej w metodzie bazowej). To gwarantuje, że klient używający referencji bazowej nie będzie zaskoczony.

35. Rzutowanie na typ klasy abstrakcyjnej

```

java

abstract class Shape { abstract double area(); }
class Circle extends Shape {
    double area() { return 3.14; }
}

Shape s = new Circle(); // ✅ upcasting – niejawny, zawsze bezpieczny
Circle c = (Circle) s; // ✅ downcasting – jawny, wymaga rzutowania
Shape s2 = (Shape) new Circle(); // ✅ rzutowanie na typ abstrakcyjny – legalne

// Rzutowanie nie tworzy nowego obiektu! To tylko zmiana "widoku" na istniejący obiekt
// Obiekt pozostaje Circle niezależnie od typu referencji.

```

Rzutowanie na typ abstrakcyjny jest **zawsze dopuszczalne syntaktycznie**. W czasie wykonania sprawdzane jest czy obiekt faktycznie jest instancją tego typu (lub podtypu).

`ClassCastException` gdy nie jest.

To jeden z **najgroźniejszych antypatternów** w Javie:

```
java

abstract class Base {
    Base() {
        initialize(); // Wywołanie metody abstrakcyjnej w konstruktorze!
    }
    abstract void initialize();
}

class Child extends Base {
    private String value = "Hello";

    Child() {
        super(); // wywołuje Base(), który wywołuje initialize() – PRZED inicjalizacją
    }

    @Override
    void initialize() {
        System.out.println(value); // value jest NULL w tym momencie!
    }
}

new Child(); // wypisze: null (!)
```

Problem: Podczas wywołania `super()` pola `Child` jeszcze nie są zainicjalizowane. `value` ma wartość `null` (domyślna dla `String`). Przez polimorfizm `initialize()` to `Child.initialize()`, ale `Child` jest jeszcze w trakcie konstrukcji. To subtelny, trudny do wykrycia błąd.

37. Package-private metoda abstrakcyjna a klasa w innym pakiecie

```
java
```

```
// pakiet: com.example.base
abstract class Base {
    abstract void doWork(); // domyślny dostęp – package-private
}

// pakiet: com.other
class Child extends Base {
    // ✖ BŁĄD KOMPILACJI – Child nie widzi metody abstrakcyjnej doWork()
    // bo jest package-private w com.example.base, a Child jest w com.other
}
```

Skutek: klasy w innych pakietach **nie mogą dziedziczyć** po takiej klasie abstrakcyjnej (bo nie mogą nadpisać package-private metody abstrakcyjnej). Kompilator zgłosi błąd. To jest rzadki ale ważny przypadek brzegowy.

38. Dlaczego Java nie obsługuje wielokrotnego dziedziczenia klas?

Architektoniczna przyczyna (Diamond Problem):



Jeśli (B) i (C) nadpisują metodę z (A), a (D) dziedziczy po (B) i (C) — która wersja metody dziedziczy (D)? Dwuznaczność.

JVM rozwiązuje to przez:

- Każda klasa ma jedną metodę w wirtualnej tabeli metod (vtable).
- Wielokrotne dziedziczenie klas wymagałoby skomplikowanej logiki wyboru.
- Interfejsy z metodami (default) mają specjalne reguły rozwiązywania konfliktów (klasa nadrzędna wygrywa nad interfejsem; jeśli dwa interfejsy — musisz nadpisać ręcznie).

39. Kowariantny typ zwracany (covariant return type)

```
java
```



```
abstract class Animal {
    public abstract Animal create();
}

class Dog extends Animal {
    @Override
    public Dog create() { // ✅ OK – Dog jest podtypem Animal
        return new Dog();
    }
}
```

Podklasa może **zawęzić** typ zwracany do podtypu oryginalnego. To nazywa się **kowariantnym typem zwracanym** (covariant return type). Dozwolone od Java 5.

Niedozwolone:

```
java

class Dog extends Animal {
    @Override
    public Object create() { return new Dog(); } // ❌ Object jest nadtypem Animal
}
```

40. `final` klasa z metodami `abstract` — nielegalna kombinacja

```
java

// ❌ BŁĄD KOMPILACJI – final uniemożliwia dziedziczenie, abstract wymaga dziedziczenia
public final abstract class MyClass {
    public abstract void doSomething();
}
```

Kompilator natychmiast odrzuca tę kombinację. `final` + `abstract` na klasie to wewnętrzna sprzeczność logiczna — niemożliwa do zaistnienia. Nie ma żadnego obejścia tej reguły.